RoomRadar QC Test Plan Description

Project: RoomRadar QC

1. Purpose of the Test Plan

The purpose of this test plan is to verify that RoomRadar QC meets the functional and non-functional requirements defined in the requirements and specification document. The test plan checks whether users can search for available rooms, filter results by building, view room schedules, and whether admin users can securely access protected admin features.

This test plan also supports validation of the system by connecting each test case to a specific requirement through a traceability matrix.

---

2. System Under Test

RoomRadar QC is a web application that helps Queens College students find available classrooms based on course schedule data.

The system includes:

- A React/Vite frontend.
- A Flask backend.
- Course schedule data used to determine room availability.
- Student-facing room search pages.
- Room schedule details pages.
- Admin login and semester schedule upload functionality.

---

3. Testing Scope

3.1 Features Included in Testing

The following features are included in testing:

- Searching for available rooms by day and time.

- Validating required search fields.
- Preventing invalid time ranges.
- Filtering results by building.
- Searching across all buildings.
- Displaying available room cards.
- Displaying full building names.
- Opening the room details page.
- Displaying room schedules clearly.
- Supporting Monday through Sunday searches.
- Excluding online and off-campus locations.
- Displaying the room availability disclaimer.
- Protecting admin-only pages.
- Uploading semester schedule data as an admin.
- Backend room search API.
- Backend room schedule API.
- Basic usability, reliability, performance, and maintainability checks.

3.2 Features Not Included in Testing

The following features are not included because they are outside the project scope:

- Real-time room occupancy detection.
- Room reservation or booking.
- Live door lock status.
- Sensor-based or camera-based availability tracking.
- Official Queens College room scheduling system integration.

---

4. Test Environment

Testing will be performed in a local development environment.

Frontend Environment

- Framework: React with Vite
- Browser: Google Chrome
- Frontend URL: `http://localhost:5173`

Backend Environment

- Framework: Python Flask

- Backend URL: `http://127.0.0.1:5000`
- API Format: JSON

Data Environment

- Course schedule data file loaded into the backend.
- The schedule data includes day, time, building, room, and course information.

---

5. Test Data

The test data will include course schedule records with:

- Weekday classes.
- Weekend classes.
- Multiple building locations.
- Physical campus rooms.
- Online and off-campus entries.
- Rooms with scheduled conflicts.
- Rooms without scheduled conflicts.
- Valid and invalid uploaded schedule files.

Example test values:

Field	Example Value
Day	Tuesday
Start Time	12:30 PM
End Time	3:30 PM
Building	Science Building
Building Code	SB
Room Code	SB E243

---

6. Test Cases

TC1: Search by Day and Time Range

Related Requirement: FR1

Use Case: Student searches for available rooms.

Preconditions:

- Backend server is running.
- Frontend server is running.
- Schedule data is loaded.

Test Steps:

1. Open the RoomRadar QC frontend.
2. Navigate to the Find Available Rooms page.
3. Select `Tuesday` as the day.
4. Select `12:30 PM` as the start time.
5. Select `3:30 PM` as the end time.
6. Click the Search Rooms button.

Expected Result:

The system displays a list of rooms that do not have scheduled classes during the selected time range.

Actual Result:

The system displayed available rooms that did not have scheduled class conflicts during the selected Tuesday time range.

Status:

Pass

TC2: Required Field Validation

Related Requirement: FR2

Use Case: Student attempts to search without completing the form.

Preconditions:

- Frontend server is running.

Test Steps:

1. Open the Find Available Rooms page.
2. Leave one or more required fields blank.
3. Click the Search Rooms button.

Expected Result:

The system displays an error message asking the user to select a day, start time, and end time.

Actual Result:

The system displayed an error message when the user attempted to search without completing the required fields.

Status:

Pass

TC3: Invalid Time Range Validation

Related Requirement: FR3
Use Case: Student enters an invalid time range.

Preconditions:

- Frontend server is running.

Test Steps:

1. Open the Find Available Rooms page.
2. Select a valid day.
3. Select `3:00 PM` as the start time.
4. Select `1:00 PM` as the end time.
5. Click the Search Rooms button.

Expected Result:

The system prevents the search and displays an error message stating that the start time must be before the end time.

Actual Result:

The system blocked the search and displayed an error message stating that the start time must be before the end time.

Status:

Pass

TC4: Filter Results by Building

Related Requirement: FR4

Use Case: Student filters available rooms by a selected building.

Preconditions:

- Backend server is running.
- Frontend server is running.
- Schedule data includes rooms in the selected building.

Test Steps:

1. Open the Find Available Rooms page.
2. Select `Tuesday`.
3. Select a valid start time and end time.
4. Select `Science Building` from the building dropdown.
5. Click the Search Rooms button.

Expected Result:

The system displays only available rooms from Science Building. Room codes should use the `SB` building code.

Actual Result:

The system returned only available rooms from Science Building, and the displayed room codes matched the `SB` building code.

Status:

Pass

TC5: Search All Buildings

Related Requirement: FR5

Use Case: Student searches for available rooms across all buildings.

Preconditions:

- Backend server is running.
- Frontend server is running.
- Schedule data includes multiple buildings.

Test Steps:

1. Open the Find Available Rooms page.
2. Select a valid day.
3. Select a valid start time and end time.
4. Select `All Buildings`.
5. Click the Search Rooms button.

Expected Result:

The system displays available rooms from multiple supported buildings when available.

Actual Result:

The system returned available rooms from multiple supported buildings when `All Buildings` was selected.

Status:

Pass

TC6: Display Available Room Cards

Related Requirement: FR6
Use Case: Student views search results.

Preconditions:

- A successful room search has been completed.

Test Steps:

1. Complete a valid room search.
2. Review the results section.

Expected Result:

Each available room appears in a card layout. Each card includes the building name, room code, available time range, and a `View Room Schedule` button.

Actual Result:

The system displayed each available room in a card layout with the building name, room code, available time range, and `View Room Schedule` button.

Status:

Pass

TC7: Display Full Building Names

Related Requirement: FR7
Use Case: Student views readable building names.

Preconditions:

- A successful room search has been completed.

Test Steps:

1. Complete a valid room search.
2. Look at the building label on each room card.

Expected Result:

The system displays full building names such as `Science Building`, `Kiely Hall`, `Powdermaker Hall`, and `Remsen Hall` instead of only building codes.

Actual Result:

The system displayed full building names instead of only building abbreviations.

Status:

Pass

TC8: Open Room Schedule Details

Related Requirement: FR8
Use Case: Student views details for a selected room.

Preconditions:

- A successful room search has been completed.
- At least one room card is displayed.

Test Steps:

1. Click the `View Room Schedule` button on a room card.
2. Observe the page route and displayed information.

Expected Result:

The system navigates to the Room Details page and displays the selected room's schedule for the chosen day.

Actual Result:

The system successfully navigated to the Room Details page and displayed the selected room's schedule for the chosen day.

Status:

Pass

TC9: Format Room Schedule Times Clearly

Related Requirement: FR9
Use Case: Student views readable schedule times.

Preconditions:

- Room Details page is open.
- Schedule entries are available for the selected room.

Test Steps:

1. Open a room's schedule details.
2. Review the displayed start and end times.

Expected Result:

The system displays schedule times without unnecessary seconds. For example, `11:53:00` should appear as `11:53 AM` or `11:53`.

Actual Result:

The system displayed schedule times without unnecessary seconds, making the times easier to read.

Status:

Pass

TC10: Search Saturday and Sunday

Related Requirement: FR10

Use Case: Student searches for weekend room availability.

Preconditions:

- Schedule data includes Saturday or Sunday entries.
- Backend server is running.
- Frontend server is running.

Test Steps:

1. Open the Find Available Rooms page.
2. Select `Saturday`.
3. Select a valid time range.
4. Click Search Rooms.
5. Repeat the process for `Sunday`.

Expected Result:

The system processes Saturday and Sunday search requests and returns results when rooms are available.

Actual Result:

The system processed both Saturday and Sunday searches successfully and returned results when available.

Status:

Pass

TC11: Exclude Online and Off-Campus Locations

Related Requirement: FR11

Use Case: Student searches for physical rooms only.

Preconditions:

- Schedule data contains online or off-campus entries.

Test Steps:

1. Search across all buildings.
2. Review all returned results.

Expected Result:

The search results do not display `Online`, `Off Campus`, `OL`, or `OC` as available study room locations.

Actual Result:

The system excluded `Online`, `Off Campus`, `OL`, and `OC` locations from the physical room search results.

Status:

Pass

TC12: Display Availability Disclaimer

Related Requirement: FR12
Use Case: Student reads availability warning.

Preconditions:

- Frontend server is running.

Test Steps:

1. Open the RoomRadar QC home page or search page.
2. Locate the room availability disclaimer.

Expected Result:

The system displays a disclaimer explaining that room availability is based on course schedule data, not real-time occupancy.

Actual Result:

The system displayed the room availability disclaimer explaining that results are based on course schedule data and not real-time occupancy.

Status:

Pass

TC13: Protect Admin-Only Pages

Related Requirement: FR13

Use Case: Unauthorized user attempts to access admin page.

Preconditions:

- User is not logged in as an admin.

Test Steps:

1. Open the application.
2. Attempt to access the admin dashboard directly through the URL.

Expected Result:

The system prevents access and redirects the user to the admin login page or blocks the protected page.

Actual Result:

The system prevented unauthorized access to protected admin pages and redirected unauthenticated users to the admin login page.

Status:

Pass

TC14: Admin Semester Data Upload

Related Requirement: FR14

Use Case: Admin uploads semester schedule data.

Preconditions:

- Admin account exists.
- Admin user is logged in.
- Valid semester schedule file is available.

Test Steps:

1. Log in as an admin.
2. Open the admin dashboard.
3. Upload a valid semester schedule file.
4. Submit the upload.
5. Perform a room search using data from the uploaded file.

Expected Result:

The system accepts the valid file and updates the schedule data used for room searches.

Actual Result:

The admin user successfully uploaded a valid semester schedule file, and the uploaded data was reflected in room search results.

Status:

Pass

TC15: Invalid Semester Data Upload

Related Requirement: FR15
Use Case: Admin uploads invalid schedule data.

Preconditions:

- Admin user is logged in.
- Invalid schedule file is available.

Test Steps:

1. Open the admin dashboard.
2. Upload a file that does not follow the expected schedule format.
3. Submit the upload.

Expected Result:

The system rejects the invalid file or displays an error message explaining that the file format is not valid.

Actual Result:

The system rejected the invalid file and displayed an error message explaining that the file format was not valid.

Status:

Pass

TC16: Backend Room Search API

Related Requirement: FR16

Use Case: Backend returns available rooms from API request.

Preconditions:

- Backend server is running.
- Schedule data is loaded.

Test Steps:

1. Open a browser or API testing tool.
2. Send a GET request to:
   `/api/rooms/search?day=Tuesday&starttime=12:30&endtime=15:30&building=SB`
3. Review the response.

Expected Result:

The backend returns a JSON list of available rooms matching the search criteria.

Actual Result:

The backend returned a valid JSON list of available rooms matching the provided search criteria.

Status:

Pass

TC17: Backend Room Schedule API

Related Requirement: FR17

Use Case: Backend returns schedule for a selected room.

Preconditions:

- Backend server is running.
- A valid room ID exists.

Test Steps:

1. Open a browser or API testing tool.
2. Send a GET request to:
   `/api/rooms/{id}/schedule?day=Tuesday`
3. Review the response.

Expected Result:

The backend returns a JSON list of schedule entries for the selected room and day.

Actual Result:

The backend returned a valid JSON list of schedule entries for the selected room and day.

Status:

Pass

---

7. Non-Functional Test Cases

NFT1: Usability Test

Related Requirement: NFR1

Test Steps:

1. Ask a tester to open the Find Available Rooms page.
2. Ask the tester to search for an available room without additional instructions.
3. Observe whether the tester can complete the task.

Expected Result:

The tester can complete a room search without confusion or help from the development team.

Actual Result:

The tester was able to complete a room search without confusion or help from the development team.

Status:

Pass

NFT2: Responsiveness Test

Related Requirement: NFR2

Test Steps:

1. Open the application in a browser.

2. Resize the browser window.
3. Test the page at laptop and smaller screen widths.

Expected Result:

The layout remains readable and usable.

Actual Result:

The layout remained readable and usable when tested at laptop and smaller browser widths.

Status:

Pass

NFT3: Performance Test

Related Requirement: NFR3

Test Steps:

1. Open the Find Available Rooms page.
2. Perform a room search.
3. Observe the time it takes for results to appear.

Expected Result:

Room search results appear within a few seconds.

Actual Result:

Room search results appeared within a few seconds.

Status:

Pass

NFT4: Maintainability Review

Related Requirement: NFR4

Test Steps:

1. Inspect the project folder structure.
2. Confirm that frontend and backend files are separated.

Expected Result:

The repository is organized into separate frontend and backend sections.

Actual Result:

The repository was organized into separate frontend and backend sections.

Status:

Pass

NFT5: Security Review

Related Requirement: NFR5

Test Steps:

1. Inspect the GitHub repository.
2. Confirm that Firebase service account keys and private credentials are not committed.
3. Review `.gitignore`.

Expected Result:

Private keys and credential files are excluded from the repository.

Actual Result:

Private keys and credential files were excluded from the repository using `.gitignore`.

Status:

Pass

NFT6: Reliability Test

Related Requirement: NFR6

Test Steps:

1. Stop the backend server.
2. Attempt to perform a room search from the frontend.
3. Observe the frontend behavior.

Expected Result:

The frontend displays an error message instead of crashing.

Actual Result:

The frontend displayed an error message instead of crashing when the backend server was unavailable.

Status:

Pass

---

8. Traceability Matrix

Requirement ID	Requirement Description	Related Test Case
FR1	Search by day and time range	TC1
FR2	Validate required search fields	TC2
FR3	Validate time range	TC3
FR4	Filter by building	TC4
FR5	Search all buildings	TC5
FR6	Display available room cards	TC6
FR7	Display full building names	TC7
FR8	View room schedule details	TC8
FR9	Format schedule times clearly	TC9
FR10	Support weekday and weekend searches	TC10
FR11	Exclude online and off-campus locations	TC11
FR12	Display availability disclaimer	TC12
FR13	Protect admin-only pages	TC13
FR14	Admin semester data upload	TC14
FR15	Validate uploaded semester data format	TC15
FR16	Backend room search API	TC16
FR17	Backend room schedule API	TC17
NFR1	Usability	NFT1

Requirement ID	Requirement Description	Related Test Case
NFR2	Responsiveness	NFT2
NFR3	Performance	NFT3
NFR4	Maintainability	NFT4
NFR5	Security	NFT5
NFR6	Reliability	NFT6

---

9. Use Case to Test Case Mapping

Use Case ID	Use Case Name	Related Requirements	Related Test Cases
UC1	Search for Available Rooms	FR1, FR2, FR3, FR4, FR5	TC1, TC2, TC3, TC4, TC5
UC2	View Room Results	FR6, FR7, FR11, FR12	TC6, TC7, TC11, TC12
UC3	View Room Schedule Details	FR8, FR9, FR17	TC8, TC9, TC17
UC4	Search Weekend Availability	FR10	TC10
UC5	Admin Login and Protected Access	FR13	TC13
UC6	Admin Uploads Semester Data	FR14, FR15	TC14, TC15
UC7	Backend API Room Search	FR16	TC16

---

10. Sequence Diagram References

The following sequence diagrams can be used to support the test plan.

SD1: Student Searches for Available Rooms

Related use case: UC1
Related requirements: FR1, FR2, FR3, FR4, FR5
Related test cases: TC1, TC2, TC3, TC4, TC5

Basic flow:

1. Student opens the Find Available Rooms page.
2. Student selects day, start time, end time, and building.
3. Frontend validates the input.
4. Frontend sends a request to the backend room search API.
5. Backend checks the schedule data.
6. Backend returns available rooms.
7. Frontend displays room cards.

SD2: Student Views Room Schedule Details

Related use case: UC3
Related requirements: FR8, FR9, FR17
Related test cases: TC8, TC9, TC17

Basic flow:

1. Student clicks `View Room Schedule`.
2. Frontend navigates to the Room Details page.
3. Frontend sends a request to the backend room schedule API.
4. Backend retrieves the room schedule for the selected day.
5. Backend returns schedule entries.
6. Frontend formats and displays the schedule times.

SD3: Admin Uploads Semester Schedule Data

Related use case: UC6
Related requirements: FR13, FR14, FR15
Related test cases: TC13, TC14, TC15

Basic flow:

1. Admin opens the admin login page.
2. Admin logs in.
3. System verifies admin authentication.
4. Admin opens the admin dashboard.
5. Admin uploads the semester schedule file.
6. Backend validates and processes the uploaded file.
7. System updates the schedule data source.
8. Student searches use the updated semester data.

---

11. Test Completion Criteria

Testing is considered complete when:

- All functional test cases have been executed.
- All non-functional test cases have been reviewed.
- Each requirement has at least one matching test case.
- Major defects have been fixed or documented.
- Any remaining limitations are included in the software limitations document.
- Screenshots or result files are saved for validation.

---

12. Test Result Documentation

Test results should be saved in the `result_files` folder.

Suggested result files include:

- Screenshot of successful room search.
- Screenshot of no results found.
- Screenshot of invalid time range validation.
- Screenshot of room details schedule page.
- Screenshot of admin login page.
- Screenshot of admin upload result.
- API response screenshots or saved JSON outputs.
- Completed test case table with pass/fail status.
